Static Wears: E-Commerce Platform

A PROPOSAL PRESENTED BY

J.K.D.C.D.JAYANETHTHI

(AMP/IT/2324/F/036)

Supervisor:

Ms.T.H. DINUSHA HEWAGE

in partial fulfillment of the requirement for the award of the HND of

INFORMATION TECHNOLOGY

HARDY ATI

SRI LANKA

2026

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATION

ER	Entity Relationship
IDE	Integrated Development Environment
UI	User Interface
ATI	Advance Technological Institute.
SLIATE	Sri Lanka Institute of Advance Technological Education

CHAPTER 1
INTRODUCTION

The way we buy things has changed so much that a “working” website is other than enough. In this day and age, if a site is down or slow during a sale, people simply close the tab and head elsewhere. Many e-commerce websites are built on one big chunk of code that developers today refer to as a “monolith”. Well, the issue with that is if one little thing breaks, everything breaks. I would like to thus approach my project differently. Using Microservices to build a Fulfilling Platform.

What I’m basically doing is separating the site into small, self-sufficient chunks instead of one large app, one deals with the products, others with the orders and another one with the users. That way, if the “email service” has a bug, people can still browse and purchase products without any problem. You can see how this makes the whole thing a lot more reliable, and way easier to fix/update later on.

I would like to make a complete system with a shop for the customers, and a dashboard for the owner. I want the customer experience to flow easy searching, a quick cart, and a secure checkout. When you are the owner, I do want to create a dashboard that does show the reality of business status and reporting how much they have sold, what is in stock. By eliminating manual work and legacy coding, I’m looking to create a tool that can handle real world traffic.

1.1 PROBLEM STATEMENT

The fact is, lots of small businesses are behind in the times. You would be surprised that there are even shops which work on paper ledgers or simple excel sheet to manage their inventory. It is a mess. It is slow and very easy to lose files, and people slip up all the time. Even the ones with their own websites usually complain that their sites become slow or even crash when too many visitors arrive at once.

Here are the key problems I want to address:

- Crashing sites: The vast majority of basic sites are not designed to cope with large numbers of people at once.
- Inadequate security: What's scary is the number of sites that do not have appropriate safeguards around data and payments.
- Human error: Entering every sale by hand is a time sink and creates inaccurate stock counts.
- Lack of real insights: most owners have no idea what product is actually driving their best business without hours of manual math juxtaposed to sales records.
- Hard to change: In a traditional site, adding a new feature is like playing Jenga, you might pull one piece and the whole thing falls over.

1.2 FUNCTIONAL REQUIREMENTS

There are two different kinds of people that the system has to do specific things: The shoppers and shop owners. I have summarized these into two tables here so it is clearer what each part of the app will take care of.

Category	What the user can do
Account	Sign up, log in securely, and manage their own profile.
Browsing	Search for items, filter by price or category, and short them.
Product Page	See high-quality photos, read descriptions, and check prices.
Cart	Add items, change quantities, and see the total price update live.
Checkout	Put in shipping info and pay safely through a secure gateway.
Updates	Get an email when they buy something or when their order ships.

Table 1: Client-sidefunctional needs

Category	What the user can do
Overview	See a main page with total sales, order counts and visitor stats.
Products	Add new items, change prices and update stock levels easily.
Orders	View every order and change the status.
Users	See who is registered and manage their accounts if needed.
Reports	Get clear data on what's selling best and how much profit is coming in.

Table 2: Admin dashboard requirements

1.3 AIM

I am here to create a professional class e-commerce site that will not only look good but perform really well in high traffic situations. I want to use a microservices architecture in order to able to show case that actually student can build and equally stable system as big companies use it. My priority is also speed, security and ease of building upon this site in future.

1.4 OBJECTIVE

To get this project done right, I've set these specific targets:

- Build the backend: Set up independent services for things like products and order using Node.js
- Create the frontend: Use Next.js and Tailwind CSS to make the site look great and work fast on mobile and desktop.
- Secure the app: Intergrade Clerk to handle logins so that user data is always protected.
- Connect the services: Use Kafka to let the difference parts of the app "talk" without slowing things down.
- Manage data properly: Use the right database for each job, PostgreSQL for structured data and MongoDB for flexible order info.
- Add an Admin Panel: Build a dashboard that gives the owner real-time charts and easy control over the shop

CHAPTER 2

LITERATURE REVIEW AND STUDY OFSIMILAR APPLICATIONS

2.1 LITERATURE REVIEW

A lot has changed in online shopping. It started with simple pages where you could barely see a picture. Now, it has grown into huge platforms that often know what you want to buy before you do. The study found that people leave a page if it takes more than a few seconds to load. And that's why speed is so important these days.

We know for a fact that the big debate in Web development as of lately is "Monolith vs Microservices". A monolith is essentially a huge toolbox where everything is welded together. It's easy to carry at first, but if you want a better hammer, you have to replace the whole box. Microservices are bit more of modular you can change the screwdriver or the wrench whenever without touching everything else. Microservices are what big companies like Amazon and Netflix use because they can make changes to their sites a thousand times a day without them going down.

On the frontend side, there are tools such as Next.js are the new standard. They use "Server - Side Rendering" in other words, the server does all of that heavy work so that an end-user's cellphone doesn't have to. This makes it so the site itself feels instant. Tailwind CSS Allows developers to create custom designs, which are much quicker than the old-fashioned method of writing long messy CSS files.

In fact, for security reasons, a lot of developers are trying to stop building their own login systems. It's pretty cheap to make mistakes and let hackers in, way more costly than if we use the services like Clerk or Auth0, both that are focused on security, gives us the ability to focus back on actually features of the shop.

2.2 STUDY OF SIMILAR APPLICATION

If you look at existing apps, you can see where they succeed and where they fail.

Shopify, and other such “out of the box” solutions are perfect to get started with, but can get pricy and you don’t have full access to your code. This all-in-one microservices platform is allowing to have the best of both worlds, a professional system’s power and the custom code flexibility.

Feature	Old “Monolith” Way	New “Microservices” Way
Stability	If one-part breaks, the whole site dies.	If one service fails, the rest keeps working.
Scaling	You have to pay for more power for the whole app.	You only scale the part that needs it (like the search)
Updates	Slow and risky.	Fast and easy to do one piece at a time
Tech Choice	You’re stuck with one language for everything.	You can use the best tool for each specific job.

Table 3: Monolith vs Microservices Comparison

CHAPTER 3
MATERIALS AND METHODS

3.1 MATERIALS

3.1.1 HARDWARE REQUIREMENTS

- Processor – Intel i5 or i7 (or Ryzen equivalent).
- Ram – 8GB or above.
- Storage – 512GB
- Internet – Stable connection.

3.1.2 SOFTWARE REQUIREMENTS

- Windows / Mac / Linux – The main operating system for coding
- VS Code / WebStorm – Primary editor for writing all the code.
- Node. js – The engine that will run the backend services.
- Docker – To run the databases in “containers”.
- Git – To keep track of my code changes and not lose work.
- Postman – To test the APIs and make sure they return the right data.

3.2 METHODOLOGY

I will be using a mix ofWaterfall and Agile methods. To be clear about what am I building, start with a plan (Waterfall), but then hatch the app in small cycles (Agile). That way I can test each bit as I progress and correct things right away aren't working.

- Planning: Figuring out all the features and drawing how the app will look.
- Design: Setting up the database structures and deciding how the services will talk to each other.
- Building: Coding the backend first, then the frontend.
- Testing: Trying to "break" the site to find bugs and fixing them.
- Finishing: Deploying the site and writing the final report.

CHAPTER 4

TECHNOLOGIES

4.1 FRONTEND AND BACKEND PROGRAMMING

4.1.1 FRONTEND PROGRAMMING

- Next.js: This is the “brain” of the frontend. It makes the site fast and helps with Google search rankings.

- Tailwind CSS/ ShadCN: This is for the styling. It makes it easy to create a site that looks good on a phone and a laptop

4.1.2 BACKEND PROGRAMMING

- Express.js / Node.js: These are the frameworks I’ll use to build the actual services.

4.2 MICROSERVICES ARCHITECTURE

The app will be split into these pieces:

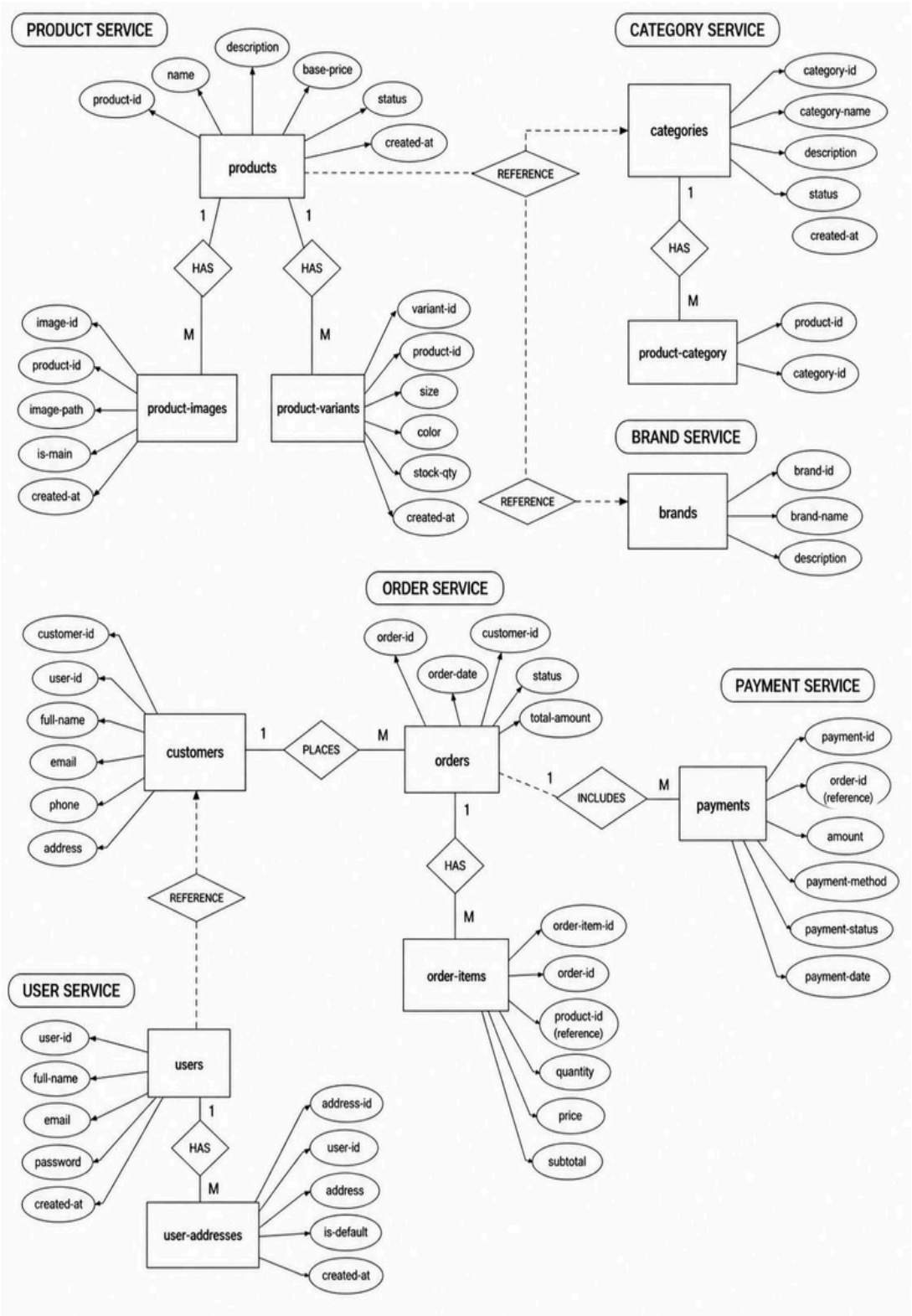
- Auth: Handles users and security.
- Products: Manages all the items for sale.
- Orders: Takes care of the buying process.
- Payments: Connects to things like Stripe to handle money.
- Email: Sends out notifications automatically.

4.3 DATABASE

Uses a polyglot persistence strategy as specified by each microservice, if required, it gets its own database.

User Service, order Service and Product Service use PostgreSQL for structured, relational data such as user accounts, product details, and categories that are best served by strict schema enforcement principle and ACID transactions.

4.3.1 ER DIAGRAM



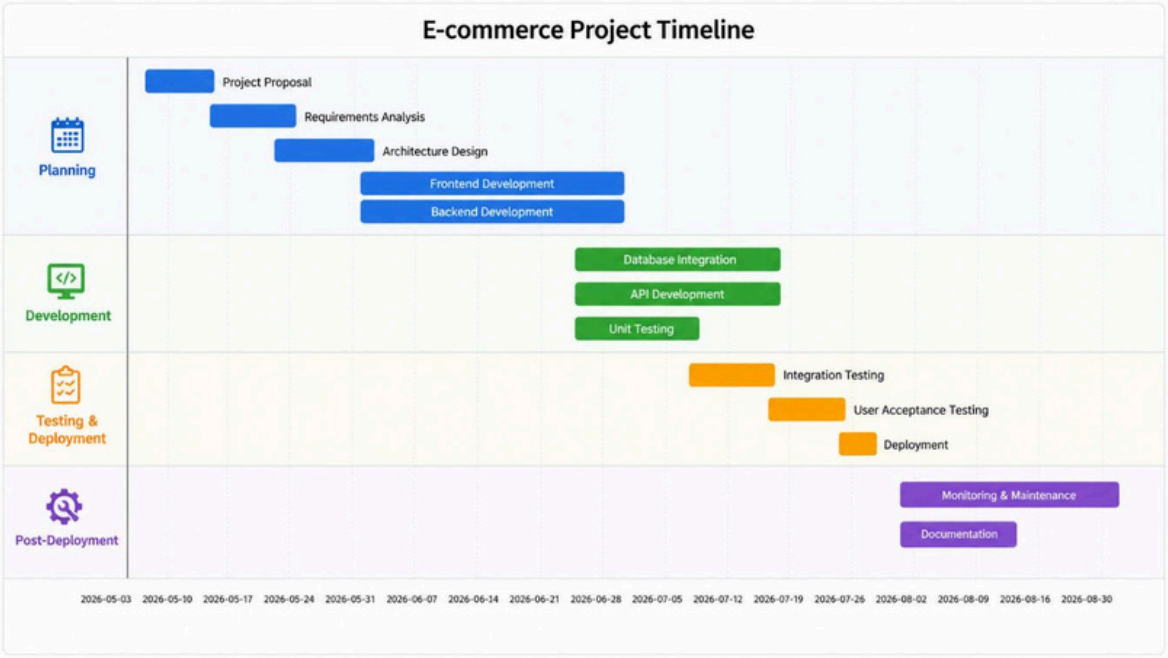
4.4 IDEs AND TOOLS

- VS Code / WebStorm
- Docker
- Kafka

CHAPTER 5

TIME PLAN

5.1 GANTT CHART



5.2 SCHEDULE AND BUDGET SUMMARY

5.2.1 SCHEDULE

Item	Start Date	End Date	Duration (Days)
Project Proposal	2026-04-20	2026-04-26	7
Requirements Analysis	2026-04-27	2026-05-07	10
Architecture Design	2026-05-08	2026-05-17	10
Frontend Development	2026-05-18	2026-06-26	40
Backend Development	2026-05-18	2026-06-26	40
Database Integration	2026-06-27	2026-07-11	15
API Development	2026-06-27	2026-07-16	20
Unit Testing	2026-06-27	2026-07-06	10
Integration Testing	2026-07-07	2026-07-16	10
User Acceptance Testing	2026-07-17	2026-07-26	10
Deployment	2026-07-27	2026-07-31	5
Monitoring & Maintenance	2026-08-01	2026-08-30	30
Documentation	2026-08-01	2026-08-15	15

Table 4: Project Schedule

5.2.2 BUDGET SUMMARY

Item	Date	Cost
Project Proposal	2024/03/16	5000
Requirements Analysis	2024/03/17 to 2024/03/25	15,000
Architecture Design	2024/03/26 to 2024/04/01	20,000
Frontend Development	2024/04/02 to 2024/04/06	40,000
Backend Development	2024/04/07 to 2024/04/09	40,000
Database Integration	2024/04/10 to 2024/04/20	15,000
API Development	2024/04/21 to 2024/04/30	18,000
Unit Testing	2024/05/01 to 2024/05/10	8,000
Integration Testing	2024/05/11 to 2024/05/16	8,000
User Acceptance Testing	2024/05/17 to 2024/05/25	8,000
Deployment	2024/05/26 to 2024/05/26	5,000
Monitoring & Maintenance	2024/05/27 to 2024/05/28	20,000
Documentation	2024/05/29 to 2024/06/01	10,000
Total Cost		LKR 212,000

Table 5: Budget Summary

CONCLUSION

Creation of this Modern Scalable E-Commerce Platform is a monumental and timely task that will be addressing the inefficiencies of traditional retail management systems. With carefully designed microservices architecture, the project is set to provide an application that is comprehensive from a functional perspective but inherently resilient, highly scalable and future-ready. By strategically embedding advanced frameworks next.js for a dynamic frontend on the data and then providing you withapowerful, an event-driven communication through Apache Kafka, enabling independentbackend services to communicate effectively, powering a high-performance environment.Thisarchitecture is custom-built to handle the demanding nature of today's rapid and reactivedigital economy, minimizing latency in user experience, ensuring consistency of data acrosschannels, and optimizing efficiency.

Once you complete this, this e-commerce systemwill basically be the strong and versatile foundation any business needs to successfullycreate or improve a well-thought-out, secure and competitive online presence. This will givebusinesses the agility to adapt to changes in the market, scalability to manage growth andreliability to guarantee that their operations never stop, ultimately providing a better userexperience for customers and continual commercial success in the digital environment.

REFERENCES

[1] S. Terdal, P. R. G., V. Mahajan, and V. S. K., "Microservices Enabled E-Commerce Web Application," *International Journal for Research in Applied Science and Engineering Technology (IJRASET)*, vol. 10, 2022. [Online]. Available: https://doi.org/10.22214/ijraset.2022.45791 (accessed Apr. 25, 2026).

[2] I. Asrowardi, S. D. Putra, and E. Subyantoro, "Designing Microservice Architectures for Scalability and Reliability in E-Commerce," in *Journal of Physics: Conference Series*, vol. 1450, no. 1, p. 012077, IOP Publishing, Feb. 2020.

[3] V. Di Francesco, "Architecting Microservices," in *Proc. IEEE International Conference on Software Architecture Workshops (ICSAW)*, pp. 224–225, 2017. https://doi.org/10.1109/ICSAW.2017.65

[4] M. Thalor et al., "Analysis of Monolithic and Microservices System Architectures for an E-Commerce Web Application," *International Journal of Intelligent Systems and Applications in Engineering*, 2024. [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/6627 (accessed Apr. 25, 2026).

[5] N. N. Nayim et al., "Performance Evaluation of Monolithic and Microservice Architecture for an E-commerce Startup," ResearchGate, Dec. 2023. [Online]. Available: https://www.researchgate.net/publication/378536265 (accessed Apr. 25, 2026).

[6] Lama Dev, "Full-Stack E-Commerce App with Microservices Architecture | Monorepo E-Commerce App Full Course," *YouTube*, 2025. [Online]. Available: https://www.youtube.com/watch?v=... (accessed Apr. 25, 2026).

[7] C. O. Oyeniran, A. O. Adewusi, A. G. Adeleke, L. A. Akwawa, and C. F. Azubuko, "Microservices Architecture in Cloud-Native Applications: Design Patterns and Scalability," *Computer Science & IT Research Journal*, vol. 5, no. 9, pp. 2107–2124, 2024.

[8] V. K. Singh, "Orchestrating Microservices with Kubernetes: Enhancing Fault Recovery and Scalability," *Journal of Cloud Computing*, vol. 12, no. 3, pp. 112–125, 2023.